Flood Detection Case Study Rubric

DS 4002 - Spring 2026 - Nehemiah Kim

Due:
See Canvas Submission format: Upload link to GitHub repository on UVA Canvas

General Description:
Submit to Canvas a link to your GitHub repository.

Why am I doing this?
This case study allows you to apply deep learning to a real-world humanitarian problem. You will train a semantic segmentation model on aerial imagery of flood-damaged scenes and evaluate how well it can automatically detect flooded regions pixel by pixel. Working through this assignment will expose you to the full pipeline of a computer vision project, from raw imagery and preprocessing to model training, evaluation, and communication, in a context with direct applications to disaster response robotics.

What am I going to do?
The GitHub repository for this case study can be found at https://github.com/Nemo0men/DS4002-CS2/tree/main. The dataset can be obtained using the link in data/data.md. You will complete the following steps:

- Download the FloodNet dataset: Use the Google Drive link in data/data.md to obtain the labeled UAV image-mask pairs.
- Use the template for EDA and first run: Run the provided baseline notebook (segformer_exp0_flood_detection.ipynb). Examine class distributions, image resolution, and the balance between flooded and non-flooded pixel counts in the masks.
- Preprocess the imagery: Resize all images to 256x256. Normalize using ImageNet mean and standard deviation. Convert multi-class masks to binary labels (flooded = 1, non-flooded = 0).
- Train a segmentation model: Choose at least one architecture from U-Net, DeepLabV3+, or SegFormer. Train using combined Binary Cross-Entropy and Dice loss to handle class imbalance. Use the Adam optimizer and save the best checkpoint based on validation IoU. All example notebooks are provided in /scripts.

- Evaluate your model: Report IoU, F1-score, and pixel accuracy on the validation set. Save loss and IoU curves. Produce at least four prediction sample images showing input, ground truth, and predicted mask.
- Document and submit: Organize your GitHub repository per the formatting spec below so another student could reproduce your results by following your README.

Tips for success:

- Start with Segformer (provided in the template): it is the simplest architecture and a strong baseline for this task.
- The dataset has a heavy imbalance toward non-flooded pixels. Dice loss is your friend.
- Use Google Colab if you do not have GPU access. The provided notebooks are Colab-compatible.
- Save checkpoints based on validation IoU, not training loss.
- Look at your prediction images, not just your metrics: failure modes often only show up visually.
- Make your variable and file names interpretable. exp1_unet_lr1e4 is better than exp1.

How will I know I have succeeded?

You will meet expectations on this case study when you successfully follow and complete the criteria in the rubric below:

Spec Category	Spec Details
Formatting	● One GitHub repository submitted via link on Canvas ● Repository titled "CS3_FloodDetection" ● Repository must contain: ○ README.md ○ LICENSE.md ○ REFERENCES.md ○ scripts/ — all notebooks for EDA, training, and evaluation ○ data/ — dataset or data.md file with a link to obtain it ○ output/ — all saved plots and prediction images

README.md	● Brief project summary (2-4 sentences) ● Software, language (Python 3.x), and package requirements • Map of the repository explaining each folder and file ● Step-by-step instructions to reproduce results ● Link to the dataset if hosted externally
Source Code	● At least one Jupyter Notebook for training and evaluating a segmentation model ● Notebook must include: ○ Data loading, preprocessing, and augmentation ○ Binary mask conversion from multi-class labels ○ Model definition and training loop ○ Validation loss and IoU are logged for each epoch ○ Model checkpoint saving based on the best validation IoU ○ Reported metrics: IoU, F1-score, pixel accuracy ○ At least 4 qualitative prediction sample images ● Comments throughout so a reader can follow your process

Spec Category	Spec Details
Output	● Training and validation loss curves saved as .png ● Training and validation IoU curves saved as .png ● At least 4 prediction sample images (input/ground truth / predicted) • All files organized in the output/ folder
REFERENCES.m d	● Markdown file citing all datasets, articles, and tools used ● IEEE documentation style ● Cite at minimum: FloodNet dataset, architecture paper(s), and any tutorials referenced ● Brief annotation under each citation explaining how it helped you